

Browser Fingerprinting Laboratory Report

Passive, Active and Behavioural Fingerprinting

Mohamed Amine El Bacha

Course: Information Security

Instructor: Imane Fouad

Institution: UM6P - College of Computing

September 29, 2026

Contents

Introduction	2
Part 1 — Passive Fingerprinting	3
1.1 What the part is about	3
1.2 What we did	3
1.3 Difficulties encountered	3
1.4 Results / Evidence	3
Part 2 — Active Fingerprinting	6
2.1 What the part is about	6
2.2 What we did	6
2.3 Difficulties encountered	6
2.4 Results / Evidence	7
Part 3 — Behavioural Fingerprinting	8
3.1 What the part is about	8
3.2 What we did	8
3.3 Difficulties encountered	8
3.4 Results / Evidence	8
Part 4 — Fingerprint Identifier (Optional Mission)	10
4.1 What the part is about	10
4.2 What we did	10
4.3 Difficulties encountered	10
4.4 Results / Evidence	10
Conclusion	12

Introduction

This lab continues the previous tracking exercise, in which Alice was followed using cookies. Alice has now deleted her cookies and enabled cookie blocking. The objective of the lab is to determine whether her browser can still be recognised without storing any identifier on her machine.

In this context, **fingerprinting** means collecting small pieces of information that a browser reveals about itself and its environment (browser version, language, screen, time zone, hardware, and even the way the user types) and combining them. Each piece alone is common, but together they can form a profile that distinguishes one browser from another.

The lab links this concept to real-world cybersecurity and privacy. Trackers can use the same techniques to follow users across visits even when cookies are removed, which makes tracking harder to notice and to block. By building a small fingerprinting prototype step by step, the lab shows concretely what a website can learn about a visitor and why deleting cookies is not enough to stay anonymous.

The work is organised in four parts: passive collection from HTTP headers, active collection with JavaScript, behavioural collection from typing, and the combination of the collected features into a single hashed identifier.

Part 1 — Passive Fingerprinting

1.1 What the part is about

The goal of this part is to find out what a server can learn from a single HTTP request, without executing any JavaScript and without storing anything in the browser.

1.2 What we did

- Created the project structure (`app.py`, `templates/index.html`, `static/fingerprint.js`), set up a Python virtual environment and installed Flask.
- In the home route of `app.py`, printed the IP address and HTTP method, then added the following headers using `request.headers.get()`: `User-Agent`, `Accept-Language`, `Accept`, `Accept-Encoding`, `Referer`, `Sec-Fetch-Site`, `Sec-Fetch-Mode` and `DNT`.
- Loaded the page at `http://127.0.0.1:8000` in a normal Brave window, then in a Brave private window, and compared the values printed in the Flask console.

1.3 Difficulties encountered

The first version of `app.py` used `request.header` instead of `request.headers`, which raised the error `AttributeError: 'Request' object has no attribute 'header'`. Correcting the attribute name solved the problem.

1.4 Results / Evidence

Figure 1 shows the headers collected from the normal window. The `User-Agent` reveals a Chromium-based browser (Chrome/154) running on macOS, the preferred language is French (`fr-FR`), and the browser supports `gzip`, `deflate`, `br`, `zstd` compression. `Referer` and `DNT` are empty because the URL was typed directly and Do Not Track was not enabled, which is consistent with `Sec-Fetch-Site: none`.

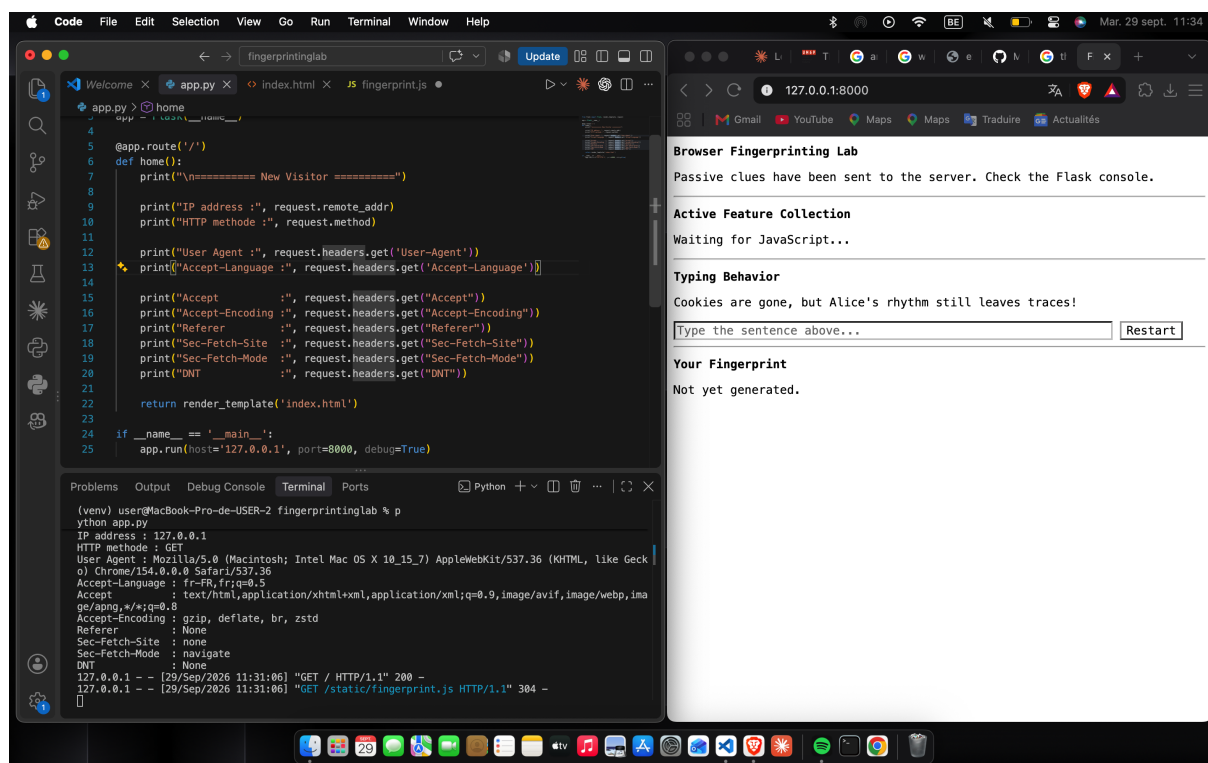


Figure 1: Passive features printed by Flask for a visit from a normal Brave window.

Figure 2 shows the same test in a private window. All headers remained identical except the language weight in `Accept-Language` ($q=0.5$ in the normal window, $q=0.7$ in the private window). Private browsing therefore did not hide the passive information sent to the server.

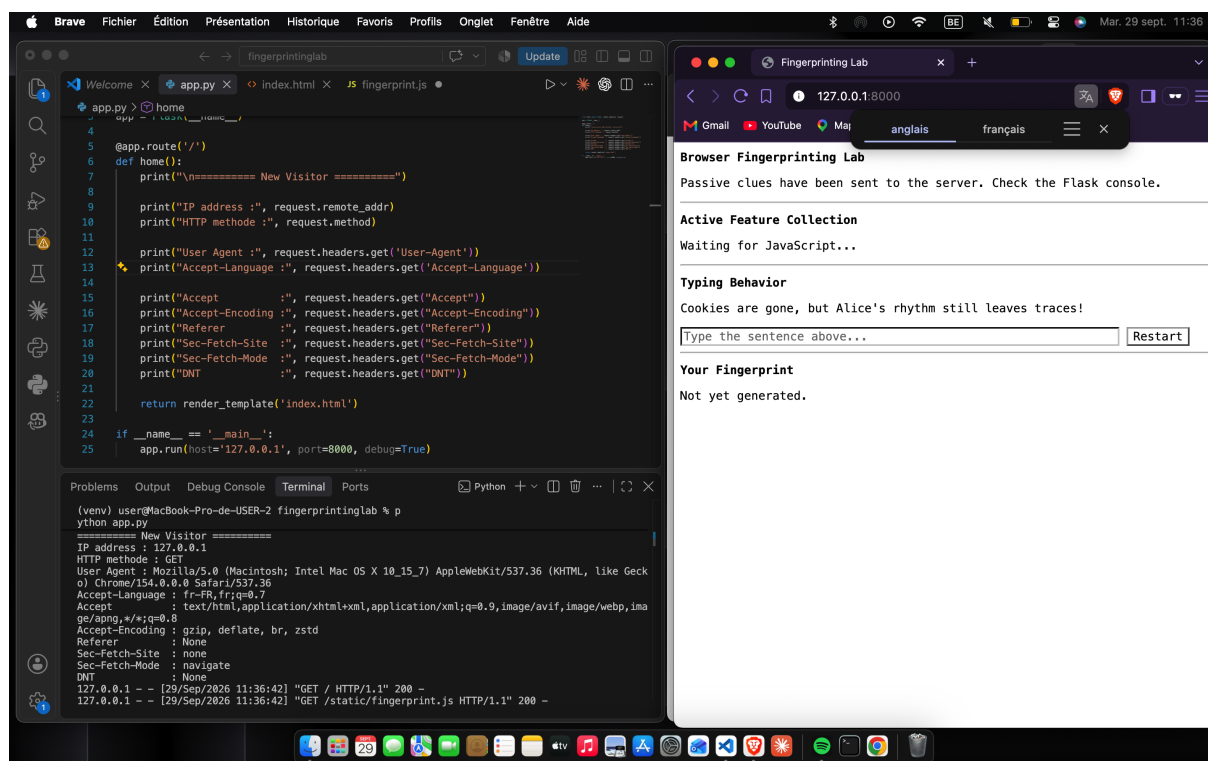


Figure 2: Passive features collected from a Brave private window. Only the Accept-Language weight changed.

Part 2 — Active Fingerprinting

2.1 What the part is about

In this part, the website executes JavaScript in the browser to query browser APIs and retrieve features that are not included in the HTTP request, then sends them back to the server.

2.2 What we did

- Added an output paragraph (`feature-output`) and the `fingerprint.js` script to `index.html`.
- In `fingerprint.js`, collected the features listed in Table 1, from the `navigator`, `screen`, `window` and `Intl` objects.
- Displayed the values on the webpage.
- Sent the values to the server as JSON with `fetch("/collect")` (POST), and added a `/collect` route in `app.py` that prints the received data.

Feature (API)	What it reveals / why it helps
<code>navigator.language(s)</code>	Browser language and language priority list; reflects the user's region and settings.
<code>screen.width/height</code>	Screen resolution; depends on the device model.
<code>screen.colorDepth</code>	Bits per pixel of the display.
<code>window.devicePixelRatio</code>	Pixel density; a value of 2 indicates a high-resolution (Retina) screen.
<code>Intl...timeZone</code>	Time zone, which gives an approximate location without asking for permission.
<code>navigator.hardwareConcurrency</code>	Number of logical CPU cores.
<code>navigator.deviceMemory</code>	Approximate RAM; not exposed by every browser.
<code>window.innerWidth/innerHeight</code>	Browser window size, which depends on how the user arranges the window.
<code>navigator.userAgent</code>	Browser and operating system, as read from JavaScript.

Table 1: Active features collected with JavaScript.

2.3 Difficulties encountered

- The `<script>` line in `index.html` contained curly quotes (`'` replaced by typographic quotes) after being pasted, which caused a `Jinja2 TemplateSyntaxError: unexpected char`. Retyping the line with straight quotes in VS Code fixed it.
- The data sent by JavaScript first received a 404 response on `POST /collect`, because the `/collect` route did not exist in `app.py` yet (visible at the bottom of

the terminal in Figure 3). Adding the route (with `jsonify` imported) solved the problem.

2.4 Results / Evidence

Figure 3 shows the active features displayed on the page: language **fr-FR**, screen 1280x800, colour depth 24, pixel ratio 2, time zone **Africa/Casablanca**, 4 CPU cores, 4 GB of device memory and a window size of 649x696. After the `/collect` route was added, the same values were received and printed by the Flask server (see the terminal in Figure 5).

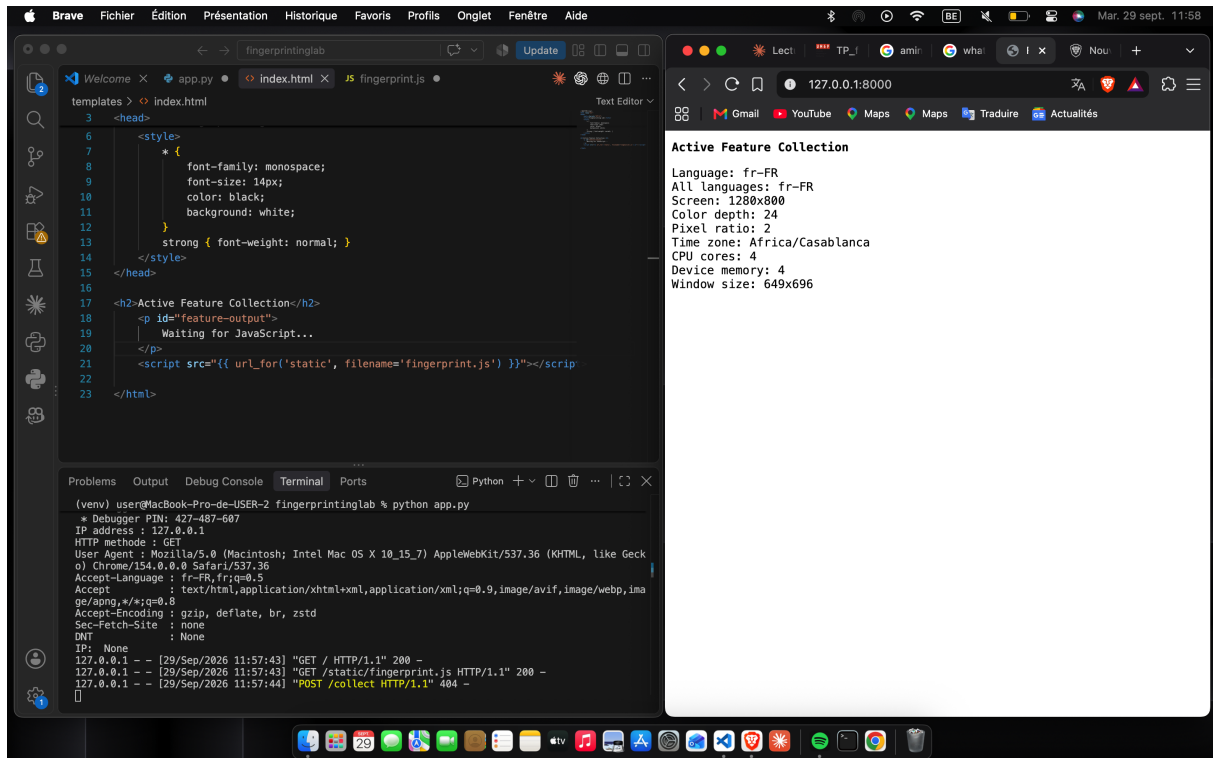


Figure 3: Active features read with JavaScript and displayed on the webpage.

Part 3 — Behavioural Fingerprinting

3.1 What the part is about

This part observes how the user interacts with the page rather than what the device is. Alice reproduces a fixed sentence, and two behavioural features are measured: her typing speed and the number of Backspace corrections.

3.2 What we did

- Added to `index.html` the target sentence (`target-sentence`), a text input (`typing-input`), a restart button (`restart-btn`) and an empty result area (`typing-results`), and included `fingerprint2.js`.
- Defined the typing speed as:

$$\text{Typing speed} = \frac{\text{number of characters in the sentence}}{\text{total typing time (s)}}$$

- Implemented the following algorithm in `fingerprint2.js`:
 1. On `keydown`: start the timer with `performance.now()` at the first key, and increment a counter when the key is `Backspace`.
 2. On `input`: compare the input value with the target sentence; when they match exactly, stop the timer.
 3. Compute the total time and the typing speed, and display them with the number of corrections.
 4. Send the time, speed and correction count to `/collect`.
 5. On `click` of the restart button: clear the input and reset all variables.

3.3 Difficulties encountered

As in Part 2, the behavioural data was first rejected with a 404 on `POST /collect` (visible in the terminal of Figure 4), since the `/collect` route had not yet been added to the server.

3.4 Results / Evidence

Figure 4 shows a completed attempt. The 57-character sentence was typed in 25.62s, giving a speed of 2.22 characters per second, with 11 corrections.

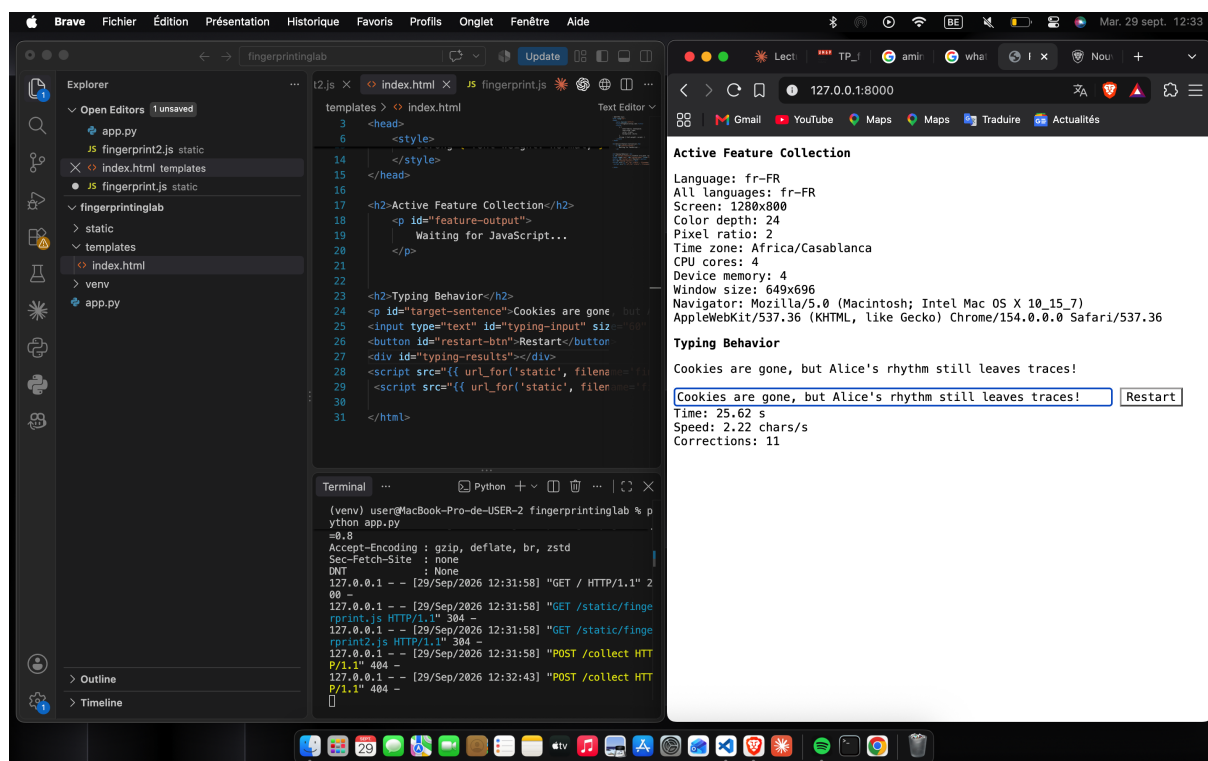


Figure 4: Typing experiment completed: total time, typing speed and number of Backspace corrections.

Part 4 — Fingerprint Identifier (Optional Mission)

4.1 What the part is about

The goal of this part is to combine the separate features into one fixed-length identifier using SHA-256, so that the server only needs to store and compare a single value.

4.2 What we did

- Combined the active features in a fixed order into one string, separated by |.
- Converted the string to bytes with `TextEncoder` and hashed it with `crypto.subtle.digest("SHA-256", ...)`.
- Converted the result into a 64-character hexadecimal string.
- Displayed the hash in `fingerprint-output` and sent only the hash to `/collect`.
- Excluded the typing features from the hash, since they change from one attempt to another.

4.3 Difficulties encountered

The hash was not received by the server at first: the requests to `/collect` returned 404 because the route was missing from `app.py`. Once the route was added, the requests returned 200.

4.4 Results / Evidence

Figure 5 shows the fingerprint displayed on the page and printed in the Flask console under “DATA RECEIVED”, together with the active features received just before it. The obtained fingerprint is:

```
c2f977fe233b886fe6f6872e3ef573cba3c7c6cf56fb20baf9a7fc82c3ad87c2
```

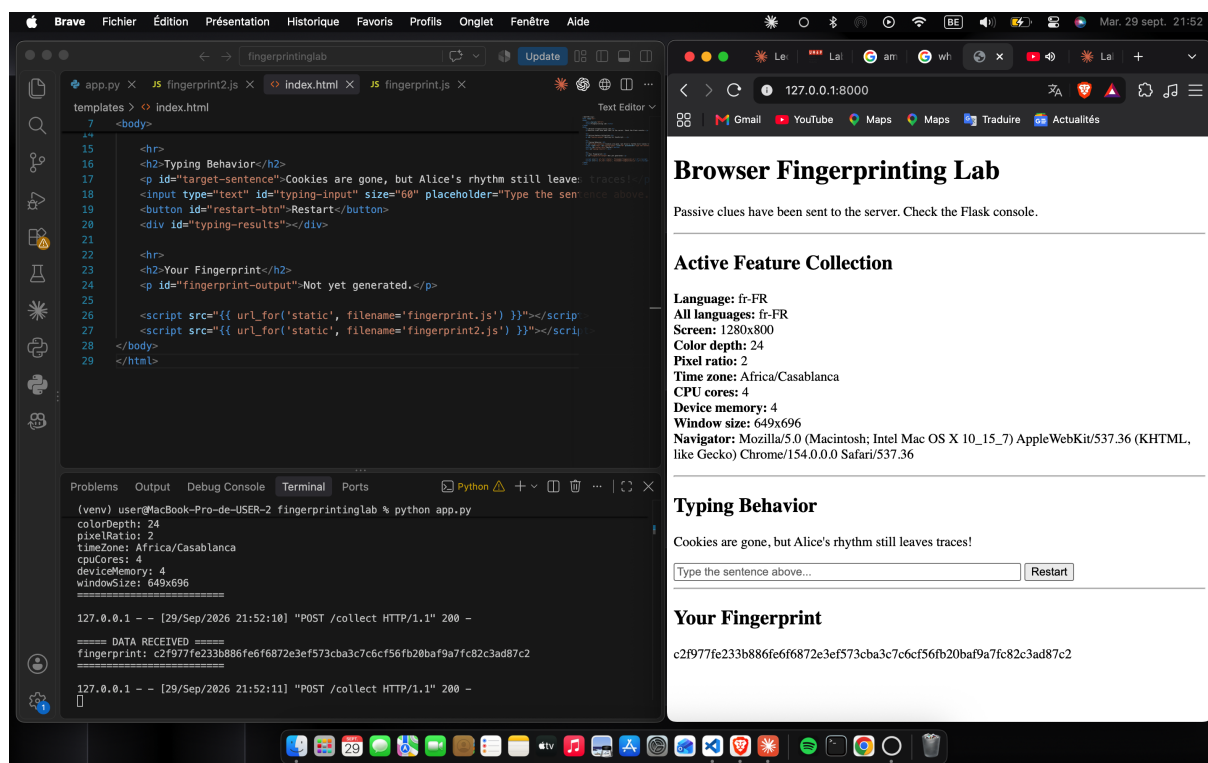


Figure 5: SHA-256 fingerprint shown on the webpage and received by the Flask server (POST /collect returning 200).

Conclusion

What we learned about tracking with fingerprinting

This lab showed that a website can still recognise a browser after cookies are deleted. Without storing anything on Alice's machine, the server collected information from three sources: the HTTP headers sent with every request, the browser properties read with JavaScript, and the way the user types. None of the active features used in this lab required any permission from the user.

The tests also showed that private browsing gives little protection against this kind of tracking. In the private window, the passive headers stayed the same except for the weight in **Accept-Language**, and the JavaScript features describe the device itself, which does not change between normal and private mode.

Finally, hashing the features into a single SHA-256 value showed how fingerprinting can stay invisible: the request only carries a short, meaningless-looking string, while the server can still compare it across visits. We also saw that not every feature is suitable for an identifier. Typing behaviour varies from one attempt to another, so it can support recognition as a probabilistic signal but cannot be part of a fixed hash.

How this work can be improved

On the fingerprinting side, the prototype could be made more distinctive and more stable:

- Add graphics-based features, such as canvas or WebGL rendering, which depend on the graphics hardware and drivers.
- Remove features that change easily from the hash, such as the window size, which changes whenever the window is resized.
- Collect more typing samples and additional behavioural features (key-hold time, inter-key interval, pauses) to build a more reliable behavioural profile.
- Test the stability of the hash by generating it several times and comparing it with the fingerprints of other students.

On the defence side, the next step is to repeat the experiment with dedicated protections, such as Brave's fingerprinting protection, Firefox's **resistFingerprinting** option, or the Tor Browser, and compare which features they hide, spoof or randomise. More generally, the most effective protections either make many users look identical or add random noise to the exposed values, so that the collected features no longer form a stable identifier.